

Embedded Operating Systems and Architectures notes

Luca Seggiani
2026

1 Lesson 23-09-26

1.1 Introduction

Computing systems are everywhere: cars, trains, aircraft, satellites, medical devices, printers, phones, industrial controllers. In an **embedded** system, we consider *software plus hardware*, not software alone: the program only makes sense together with the machine it drives. Complexity is growing dramatically. A modern car carries ≈ 100 electronic control units and on the order of 10^8 lines of on-board code, against a handful of hundreds of lines in the 1970s.

1.1.1 Operating systems

The natural question to ask is *why* an embedded system might need an operating system. The answers are many:

- Bare metal applications no longer cope with the complexity even in the embedded systems domain;
- **An operating system raises the abstraction level:** it gives application code a small set of rules and services instead of ad-hoc hardware handling;

Our goal is to connect three things that are usually taught apart: **computer architecture**, **operating systems**, and the **computer system** that has to run both.

1.1.2 Topics

The main roadmap of the course is:

- We will connect operating-system concepts to executable embedded software;
- This course lives in the **platform** layer of a computing node: from the hardware up to the system-call interface, with the application used as the thing that exercises it.

The covered topics are:

- **Minimal computing-system architecture** — juste enough to write operating systems code;
- **Bare-metal development flow** — cross-compilation, emulators and simulators, debugging and profiling, virtual platforms;
- **Operating systems for embedded systems** — real-time and process scheduling, the anatomy of an embedded OS, architecture and services of real systems (FreeRTOS);
- **Programming an embedded OS** — building it, device management and drivers, drivers for custom devices.

1.2 Introduction to embedded operating systems

An **OS** (*Operating System*) is nothing but a program, with a very simple purpose: to mediate between hardware and the programs that use it.

There are three classic ways to draw that boundary:

- **Monolithic:** kernel is one large program; drivers, filesystems, scheduling all share one address space. Both Linux and Windows fit this description.

Monolithic kernels are simple to implement once the task switching machinery is running: once we can switch to kernel mode, we can touch any resource available to the system to carry out tasks;

- **Microkernel:** kernel does the bare minimum (IPC, scheduling, basic memory); everything else runs as separate services.

Microkernels are interesting because they are inherently safer: the service handling the filesystem, for instance, cannot corrupt other services on failure (as it runs itself as a protected service with no access to other resources). The problem comes up when complex tasks have to be performed: the main overhead is given by the IPC and scheduling facilities which sit inbetween services;

- **Flat:** no kernel/user separation at all; a small set of statically-defined tasks share the CPU under one scheduler.

This is important to us because embedded systems often don't feature protection systems, and as we know complete protection is impossible without hardware support.

The core of the operating system is basically the *scheduler*: everything else can be seen as a task either the operating system or the user programs carry out, based on the resources allocated by the scheduler.

1.2.1 Hardware resources

The purpose of an OS is to make hardware *usable*. Without an OS, every application must decide how to initialise the CPU, program a timer, share a UART, and recover from an interrupt. An OS supplies a small set of rules and mechanisms so application code can ask for useful services instead of repeatedly rebuilding hardware policy.

The resources are finite, especially on a small board:

- **CPU time:** which ready task runs now? We also have to take into account the fact that CPUs interact with many other devices on the same board;
- **memory:** which code and data occupy RAM and Flash?
- **devices:** who may configure a timer or consume a received byte?
- **events:** how does an interrupt become work for the right task?

1.2.2 Types of operating systems

The three labels we gave earlier describe where operating-system services execute and how strongly they are isolated.

Type	Services offered	Tradeoff
Monolithic	Together in kernel space	Direct calls, large trusted core
Microkernel	Mostly as separate servers	Isolation, IPC overhead/design
Flat	Together in one firmware image	Tiny and direct, little isolation

Monolithic A monolithic kernel puts scheduling, drivers, filesystems, and networking in one kernel address space. Services can call each other directly and quickly.

- Linux is the familiar large-scale example
- A faulty driver can corrupt kernel state
- The design is practical when the feature set is broad and tightly coupled

On a protected machine, applications normally execute outside this trusted kernel address space.

Microkernel A microkernel retains only the most fundamental mechanisms in the kernel, usually scheduling, address-space management, and inter-process communication.

Drivers and other services become separate programs that exchange messages.

- Smaller trusted computing base
- A service failure can be contained or restarted
- More interfaces and message paths to design correctly

seL4 and QNX are useful reference points for this family.

Flat Flat embedded firmware usually has one linked image and one physical address space. Tasks cooperate through shared memory and the scheduler.

There is no process loader, no virtual-memory translation, and usually no hardware-enforced boundary between tasks. This puts more pressure on software developers, as user code must be free of bugs which could potentially crash the whole operating system. This is particularly critical in systems like vehicles, medical or industrial. We know that monolithic operating systems give us an extra layer of safety, by switching to a safer kernel mode to perform critical tasks, which user code can't normally access.

1.2.3 Operating system roadmap

Where does this course sit in this hierarchy?

- **Linux** (a monolithic kernel) and *seL4/QNX* (microkernels) are the extremes we'll read about;
- This course builds **flat** first (*scratch-os*) — the simplest point on the spectrum, and the one that makes the boot-to-scheduling pipeline fully visible. *scratch-os* is intentionally small. Its value is that every important step can fit in mind;
- Then **FreeRTOS** — closer to a minimal microkernel-flavored RTOS: a small kernel, real scheduling policies, but still no memory protection between tasks by default. FreeRTOS provides established implementations of the mechanisms we first meet in *scratch-os*:

- Task creation and priorities;
- Delay and timer-driven scheduling;
- Queues, semaphores, and mutexes;
- Interrupt-to-task communication.

The comparison matters: an API call becomes easier to reason about when its underlying state transitions are no longer invisible.

1.2.4 Real-time operating systems

The **real-time** label applied to an OS is about *bounded response*, not raw speed. The idea is that we can know exactly how long a task will take before executing it. A result computed after its deadline can be a failure even when its value is otherwise correct. This is particularly useful in embedded systems which might have external (e.g. physical) constraints, in addition to technical ones.

Requirement	Example
Functional	sample a sensor correctly
Temporal	sample it every 1 ms, before the deadline
Robustness	continue safely when one input is missing

1.2.5 Embedded operating systems

Embedded operating systems usually look different than traditional ones:

- **Resource-constrained:** kilobytes of RAM/flash, not gigabytes — no room for a general-purpose kernel's overhead;
- **Real-time:** correctness often depends on *when* something happens, not just *whether* it happens;
- **No user protection needed (usually):** one vendor, one firmware image, one set of tasks — the isolation problems a desktop OS solves mostly don't apply here.
This is mainly a cost problem: it's not that car manufactures don't *want* safety in their devices, but that CPUs which don't feature protection mechanisms are usually cheaper;
- These constraints are why *scratch-os* can be simple and still be a real, working OS.

An embedded OS is selected and configured around a particular product, not an unknown future workload.

- Static task counts and fixed buffers can replace dynamic allocation;
- A simple priority scheduler can replace a general-purpose fairness policy;
- Direct register access can replace a layered device stack;
- Predictable execution is often more valuable than maximum throughput.

Small is not incomplete when it matches the system's responsibilities.

Many **MCU** (*Micro Controller Unit*) products ship one firmware image that includes startup code, drivers, the OS, and application logic. Updating that image changes the whole system.

This simplifies deployment, but it raises the bar for discipline:

- a bad pointer can damage any writable state
- a blocked high-priority task can stop an important service
- recovery and observability must be designed into the firmware

Later, we will see how scheduling and synchronisation reduce these risks.

1.2.6 Embedded systems

There are many ways we can classify **embedded systems**:

- A computer built into a larger device to do **one dedicated job** — not a general-purpose machine you install arbitrary software on;
- Hardware + software (often called *firmware*) shipped as one unit; the end user never sees a shell or a package manager;
- Everywhere: engine controllers, pacemakers, drones, washing machines, Wi-Fi routers, the microcontroller in a USB charger;
- The population of embedded processors passed the human population decades ago — most CPUs made are never in a “computer”.

We can compare embedded systems to general-purpose ones (such as modern computer or phone operating systems):

	Embedded	General-purpose
Workload	one fixed task	anything the user installs
Resources	KB of RAM/flash	GB of RAM, TB of disk
Timing	often hard deadlines	best-effort
User model	no interactive user	multi-user, multi-tasking
Cost/power	pennies, milliwatts	dollars, tens of watts

The right-hand column is what a desktop OS is built for; *scratch-os* targets the left-hand one, which is *why* it can be small.

1.3 Microcontrollers

We begin to approach the device that we'll actually need to program. A **microcontroller** or **MCU** (*Micro Controller Unit*) integrates onto a single chip what a desktop splits across a whole motherboard:

- **CPU core** — executes instructions (ours: ARM Cortex-M3);
- **RAM** — volatile working memory, tens of KB;
- **Flash** — non-volatile program storage; code runs directly from it;

- **Peripherals** — timers, UART, GPIO, ADC, all on-chip;
- One chip, one clock, one address space — no external memory bus to wire up, no BIOS, no disk.

Compared to desktop or handled general-purpose CPUs, we note that:

- A desktop CPU is *just* the core: RAM, storage, and I/O are separate chips reached over buses; an MMU gives every process its own virtual address space;
- An MCU has **one flat physical address map** shared by code, data, and every peripheral register — no MMU, no virtual memory (a Cortex-M has at most an optional MPU);
- On an MCU, “the hardware” is *memory you write to*: there is no driver layer between your code and a device register.

1.3.1 Reset routine

As an example, let’s look into what happens to a Cortex-M CPU at reset time. After reset, the Cortex-M does not discover an operating system. It reads two words from the vector table in Flash:

1. The initial value for the main stack pointer
2. The address of `Reset_Handler`.

The reset handler is our first code. It establishes the C runtime environment before calling `main()`.

1.3.2 Registers

The CPU operates on values held in registers. Memory is accessed explicitly with load and store instructions.

Register role	Meaning
Program Counter	Selects the next instruction
Stack Pointer	Locates calls, local variables, saved state
General Registers / Programmer Registers	Hold operands and addresses
Status Register	Records conditions and interrupt state

When an exception occurs, the processor automatically saves essential state on the current stack so execution can later resume.

1.3.3 Memory

We can talk about the *memory* available to an MCU. Usually there are two kinds of memory:

- **SRAM / DRAM**: not persistent, but extremely fast. Used as main program memory;

- **Flash** memory: read-only, but still very fast and fit (in embedded systems) for containing code. This is in accord with the usual definition of *Harvard Architecture*. We should note that most general purpose devices use the opposite architecture, that is *Von Neumann Architecture* (which use program memory both for code and data).

Flash keeps its contents without power. SRAM is fast writable working memory whose contents disappear on reset or power loss.

```
Flash: vector table | .text | .rodata | initial bytes for .data
SRAM:  .data        | .bss  | heap area | stack
```

This split explains why startup code must copy some bytes and clear others before ordinary C code can safely run.